OPERATING SYSTEMS · PROJECT REVIEW 1

ThrashGuard

Deep-Learning-Based Thrashing Detection

System Design · Data Pipeline · PatchTST Model · Backend · Dashboard

Presented by

Team Members:
Adish Kumar Gupta (24BCE2092) | Lakshay Sharma (24BCE2410)

INTRODUCTION

Project Summary

The Problem

When physical memory is over-committed, a Linux system spends more time moving pages between RAM and disk than doing useful work. This is thrashing. Most tools only notice it after performance has already collapsed.

Our Approach

ThrashGuard samples five OS metrics (page fault rate, swap I/O, free memory, CPU usage, run queue length) and feeds a rolling 16-step window to a PatchTST Transformer that classifies the current system state.

Expected Outcome

An early-warning classifier that labels the system Normal, Memory Pressure, Approaching Thrashing or Thrashing, shown live on a dashboard so action can be taken before throughput collapses.

Key Objectives

1. Collect five live OS metrics from a Linux system
2. Build 16-step multivariate time-series windows
3. Classify four system states with a PatchTST model
4. Serve predictions through a Flask API and dashboard

BACKGROUND

What is Thrashing?

Definition

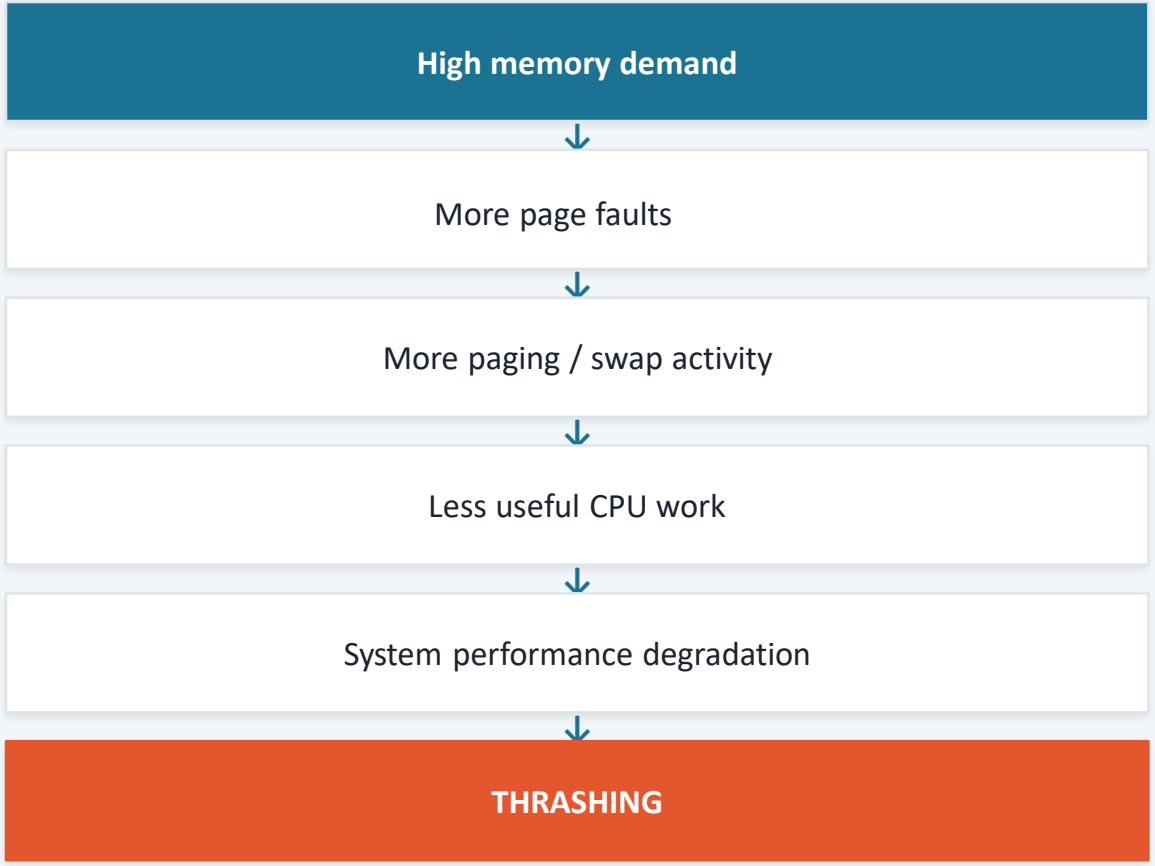
Thrashing is a state in which a system spends most of its time paging data between RAM and disk, instead of executing actual processes.

What is a Page Fault?

A page fault occurs when a process accesses a memory page that is not currently in physical RAM, forcing the OS to fetch it from disk (swap space).

Why Do Page Faults Increase?

As more processes compete for limited physical memory, each process's working set no longer fits in RAM, so page faults rise sharply and the OS must swap pages in and out continuously.



Detecting the transition toward thrashing early - rather than only after it occurs - allows the OS to intervene before throughput collapses.

Why It Matters & Limits of Current Approaches

Why detection matters

- Thrashing leaves the CPU waiting on disk, so a machine can become almost unusable while technically still running.
- Memory over-commitment is common in practice: many processes, containers or VMs share limited RAM.
- Detecting the build-up early leaves time to act (throttle, pause or terminate processes) before throughput collapses.

Goal: detect the transition toward thrashing, not just thrashing itself.

Limitations of existing approaches

1. **Static thresholds**
   Fixed cut-offs (for example on free memory) ignore trends and differ across workloads and machines.

2. **Reactive handling**
   Tools such as the Linux OOM killer act only once memory is nearly exhausted and performance has already degraded.

3. **Single-metric view**
   A high page-fault rate alone is ambiguous: it also occurs during normal program start-up.

4. **No temporal context**
   Point-in-time snapshots miss how the metrics evolve together in the lead-up to thrashing.

Literature Survey

Ref	Work	Approach / contribution	Limitation for our problem
[1]	**Denning (1968)** *Thrashing: Its Causes and Prevention*	Explains thrashing as too many active processes for the available memory; proposes working-set based load control to prevent it.	Analytical model that relies on tracking per-process working sets; no learned or predictive component.
[2]	**Weiner (2018)** *Linux PSI: Pressure Stall Information*	Kernel interface (/proc/pressure) reporting how long tasks stall waiting for CPU, memory or I/O, with trigger thresholds.	Measures current stall pressure; does not classify system state or signal that thrashing is approaching.
[3]	**Vaswani et al. (2017)** *Attention Is All You Need*	Introduces the Transformer, which uses self-attention over a sequence instead of recurrence.	Designed for language; attention cost grows quickly when every time step is a separate token.
[4]	**Nie et al. (2023)** *A Time Series Is Worth 64 Words (PatchTST)*	Splits a time series into patches used as Transformer tokens, shortening the sequence and improving long-term forecasting.	Evaluated on forecasting benchmarks; not applied to OS metrics or system-state classification.
[5]	**Chuah et al. (2025)** *Deep learning-based prediction of major page faults in cluster systems*	Applies deep learning to cluster resource-usage data and system logs to predict major page faults.	Targets page-fault prediction on HPC clusters, not graded detection of the path into thrashing.

RESEARCH GAP

Research Gap & Our Contribution

GAP 1

Existing OS tools (PSI, OOM killer, fixed thresholds) report current pressure or react late. Few offer an early-warning stage before thrashing sets in.

OUR RESPONSE

A dedicated "Approaching Thrashing" class lets ThrashGuard flag the transition, not only the final state.

GAP 2

Classical analysis and recent deep-learning work focus on single quantities such as working sets or page-fault counts, not several OS metrics read together.

OUR RESPONSE

Five metrics (page faults, swap I/O, free memory, CPU, run queue) are modelled jointly as a 16 x 5 time window.

GAP 3

Patch-based Transformers such as PatchTST are mainly benchmarked on forecasting; there is limited work applying them to OS thrashing-state classification.

OUR RESPONSE

PatchTST is adapted as a 4-class classifier on Linux metrics, with a Flask API and dashboard around it.

Problem statement: detect the build-up to thrashing early from multivariate OS metrics, using a patch-based Transformer.

Why Real OS Metric Collection?

Synthetic data (early stage)

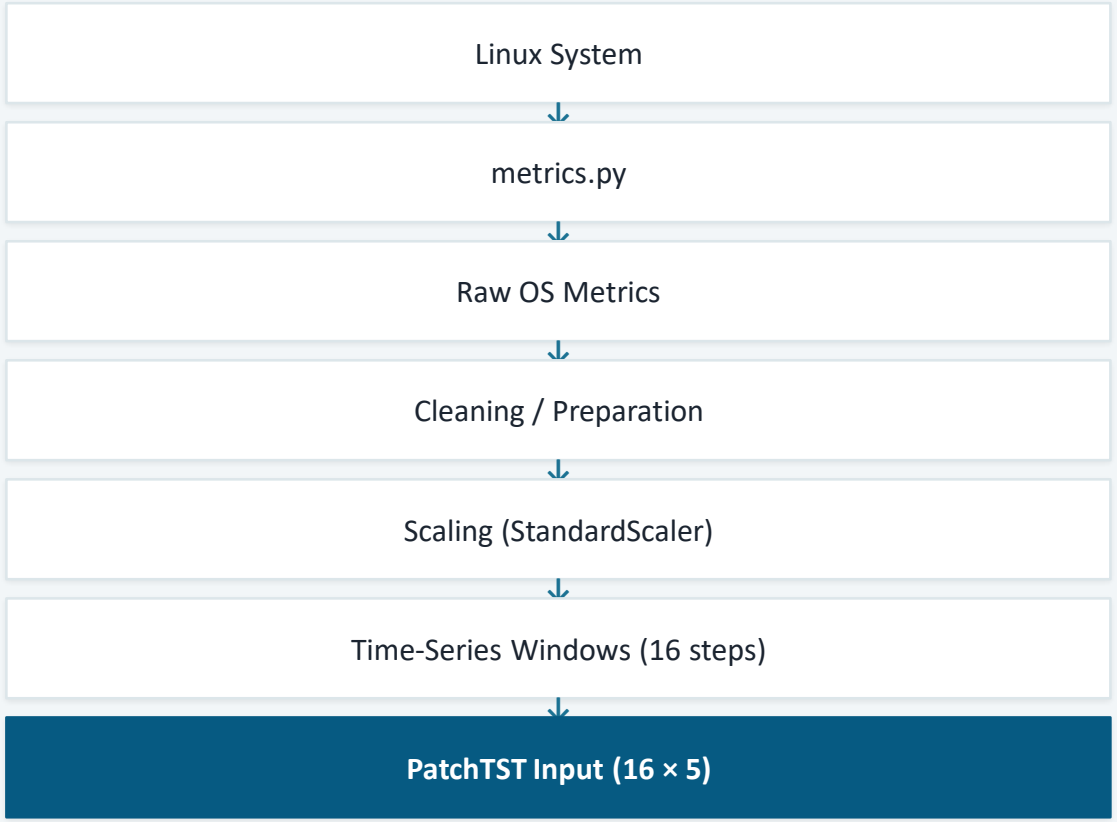
Useful for developing and testing the pipeline quickly, without needing a live system.

Limitation of synthetic data

May not fully capture the real temporal relationships between OS metrics as they occur on an actual machine.

Moving to real collection

A Linux-based metric collection approach was developed using metrics.py to gather live system readings.



Metrics & System State Labels

Input Metrics

Metric	Represents	Relevance
Page Fault Rate	Faults/sec needing disk fetch	Direct signal of memory shortage
Swap I/O	Data swapped in/out per interval	Indicates active paging pressure
Free Memory	Available physical RAM	Falling trend precedes thrashing
CPU Usage	Productive vs. idle/wait time	Drops as system stalls on I/O
Run Queue Length	Processes waiting for CPU	Rises when system is overloaded

Classification States

State	Description
0 - Normal	Healthy operation, low memory pressure
1 - Memory Pressure	Memory usage rising, faults increasing
2 - Approaching Thrashing	Sustained high paging activity
3 - Thrashing	Severe paging, CPU mostly stalled on I/O
4 - Recovery (future)	Design placeholder - not yet classified

Note: Recovery exists in the metric-collection design as a future-oriented class. The current trained model classifies only the four states above (0–3).

Why PatchTST?

The problem isn't simply "is the page fault rate high right now?" - it's how multiple OS metrics evolve together over time.

Multivariate modelling

Five metrics must be interpreted together, not as isolated numbers.

Temporal patterns

Thrashing builds up gradually - a single snapshot misses the trend.

Cross-metric relationships

Rising swap I/O alongside falling free memory is more informative than either alone.

Sequential behavior

The pattern of change leading up to and during thrashing matters, not just the current value.

What is PatchTST?

Patch-based Time Series Transformer - learns temporal patterns from multivariate sequences



What is a "Patch"?

Instead of processing every individual time step independently, PatchTST groups consecutive time steps into small patches, similar to how a vision transformer groups pixels into image patches.

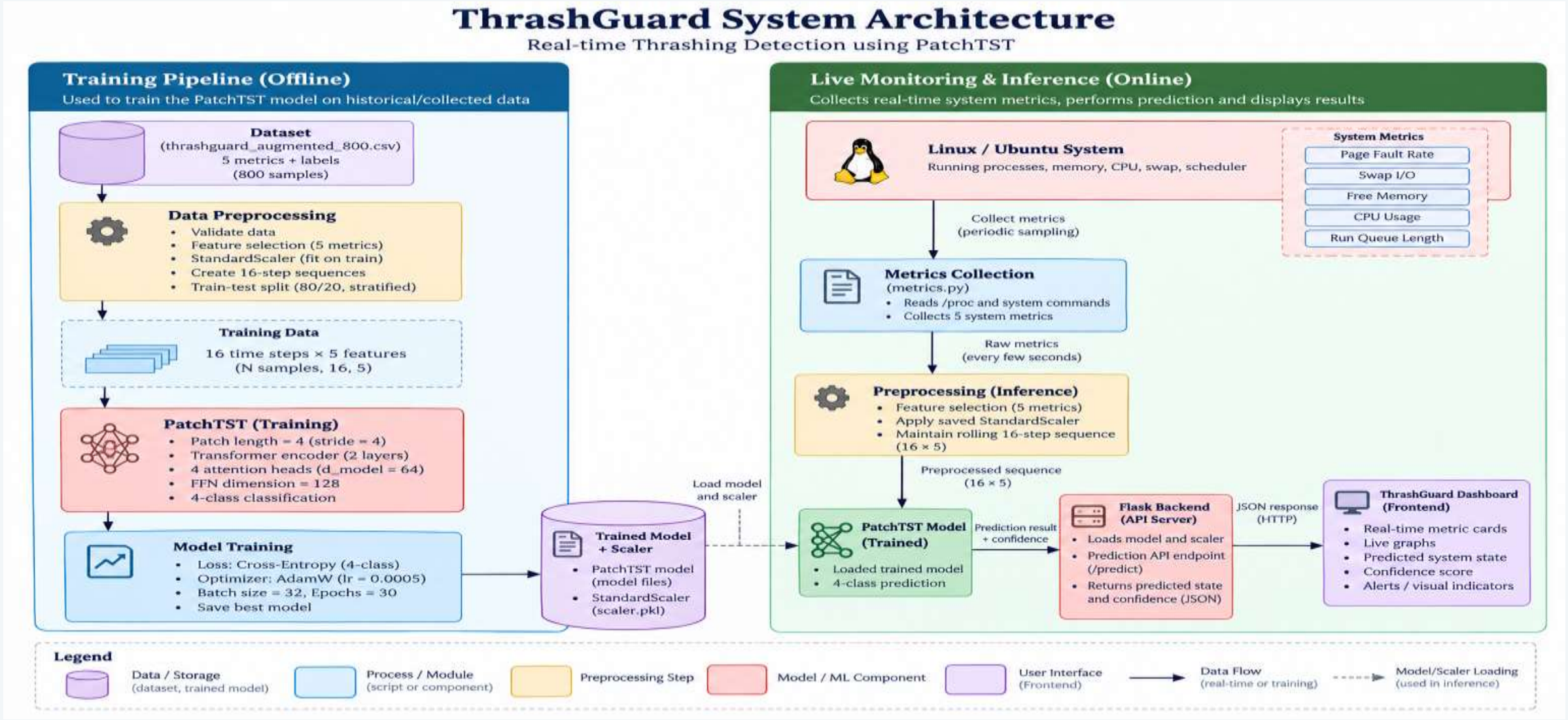
```
Time steps:   t1  t2  t3  t4  |  t5  t6  t7  t8  | ...
[t1 t2 t3 t4] → Patch 1
[t5 t6 t7 t8] → Patch 2  ...
```

(Actual configuration: patch length = 4, patch stride = 4)

Why patches help

- Reduces sequence length seen by the transformer, making attention more efficient
- Preserves local temporal structure within each patch
- Lets the model attend to relationships between patches rather than every raw point
- Well suited for a rolling 16-step sequence of 5 OS metrics

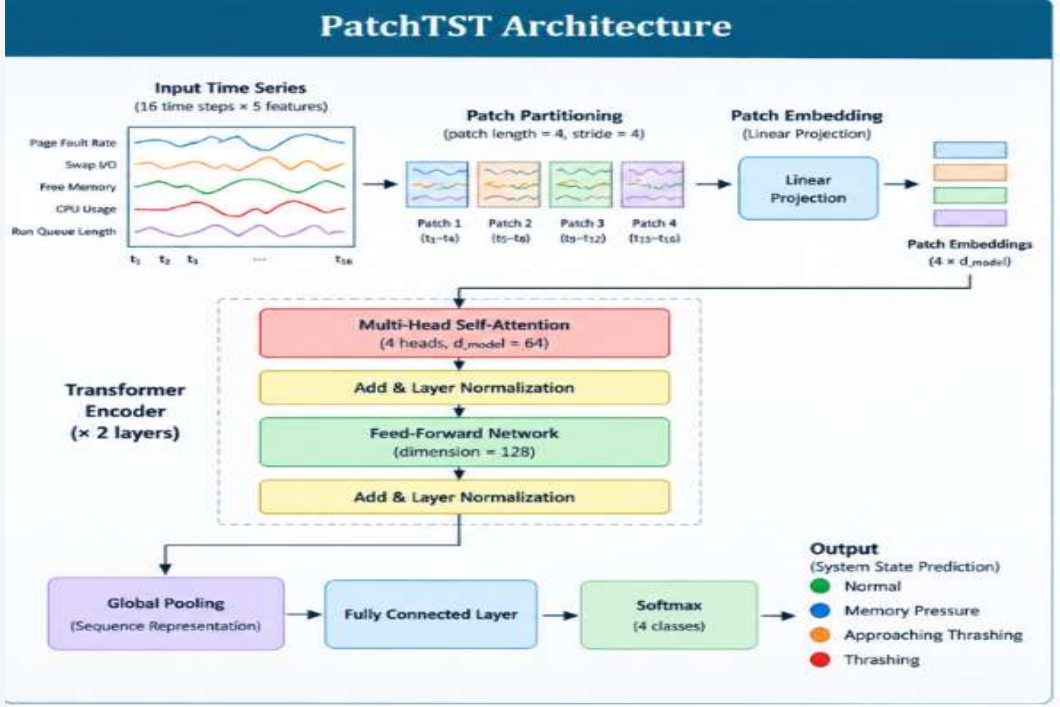
System Architecture



Model Configuration

Parameter	Value
Input shape	16 time steps × 5 features
Patch length / stride	4 / 4
Hidden layers	2
Model dimension (d_model)	64
Attention heads	4
Feed-forward dimension	128
Loss function	Cross-Entropy Loss (4-class)
Optimizer	AdamW, learning rate = 0.0005

These are the actual configuration values used in the current trained implementation.



How the model works

- Each 16 x 5 window is cut into 4 patches (length 4, stride 4).
- Patches are linearly embedded to 64 dimensions.
- A 2-layer Transformer encoder (4 heads, 128-d feed-forward) learns temporal features.
- Global pooling, a fully connected layer and softmax give 4 class probabilities.

Backend Architecture

Flask service connecting the trained model to the application

3.1 Model Loading

Loads the trained PatchTST model and the fitted StandardScaler at startup.

3.2 Input Preprocessing

Applies the saved scaler and maintains a rolling 16-sample buffer per sequence.

3.3 Prediction Pipeline

Forms the 16 × 5 sequence and passes it through the loaded model.

3.4 Evaluation

Computes accuracy, classification report, and confusion matrix on test data.

3.5 API / Prediction Output

Returns predicted class, confidence, and full class probabilities.

Endpoints: `/metrics (receive samples)` `/predict (accept sequences)` `/health` `/info`

Model Evaluation & Results

73.79%

Best observed test accuracy

Class	Accuracy
Normal	97.14%
Memory Pressure	58.33%
Approaching Thrashing	69.44%
Thrashing	71.05%

Analysis

Normal states are identified strongly. Intermediate states (Memory Pressure, Approaching Thrashing) are harder to separate because metric ranges overlap. Thrashing is more distinguishable due to stronger combined stress indicators.

Dataset

800 rows after removing Recovery; 4 balanced classes with 200 samples each.

Preparation

StandardScaler applied, 16-step windows, stratified 80/20 train-test split.

Reading the result

Intermediate classes overlap, which motivates a larger real-world dataset.

RESULTS SUMMARY

Project Completion Status

Component	Status
Thrashing concept / problem definition	Completed
Linux metric collection (metrics.py)	Completed
Data preparation (scaling, sequencing)	Completed
PatchTST model (training)	Completed
Model evaluation	Completed
Flask backend (prediction API)	Completed
Interactive frontend dashboard	Completed
Real Ubuntu → live dashboard integration	Planned next (currently simulated demo values)

Current browser demonstration uses simulated metric values; connecting live Ubuntu system metrics is the next integration step.

Conclusion

- ThrashGuard demonstrates an end-to-end pipeline: real Linux metric collection, data preparation, a PatchTST-based deep learning classifier, a Flask backend, and an interactive dashboard.
- The model classifies four system states - Normal, Memory Pressure, Approaching Thrashing, and Thrashing - with a best observed test accuracy of 73.79%.
- Recovery remains a designed-for but not yet implemented class, intended for future extension.
- The current system establishes the full working pipeline; the next phase focuses on scaling the dataset, improving accuracy, and moving from simulated to live system integration.

FUTURE SCOPE

Scope for Next Reviews

1 Bigger Dataset

Collect a larger, more diverse set of real Linux system traces to improve generalization.

2 Better Model Accuracy

Tune PatchTST hyperparameters and training strategy to improve on the current 73.79% baseline.

3 Recovery State Label

Extend the model to classify the Recovery state, currently only a design placeholder.

4 Live CPU Metric Integration

Replace simulated dashboard values with a real-time feed from live Ubuntu system metrics.

5 Intervention Policy

Add an automated response - such as adjusting process priority or admission control - to actively reduce CPU stress and prevent thrashing once detected.

References

[1] P. J. Denning, "Thrashing: Its causes and prevention," in Proc. AFIPS Fall Joint Computer Conf., vol. 33, 1968, pp. 915-922.

[2] J. Weiner, "PSI - Pressure Stall Information," Linux Kernel Documentation, 2018. [Online]. Available: https://docs.kernel.org/6.9/accounting/psi.html

[3] A. Vaswani et al., "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[4] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with Transformers," in Proc. Int. Conf. Learning Representations (ICLR), 2023.

[5] E. Chuah, A. Jhumka, S. Narasimhamurthy, and A. Ayesh, "Deep learning-based prediction of major page faults in cluster systems," CCF Trans. High Performance Computing, vol. 7, no. 5, pp. 413-430, 2025, doi: 10.1007/s42514-025-00233-2.

[6] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[7] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Proc. Int. Conf. Learning Representations (ICLR), 2019.

Thank You

Questions & Feedback

Adish Kumar Gupta (24BCE2092) | Lakshay Sharma (24BCE2410)

THRASHGUARD

A PatchTST-Based System for Memory Pressure and Thrashing Classification

Invention / Project Technical Report

Prepared by
Adish Kumar Gupta | Lakshay Sharma

Operating Systems • Time-Series Deep Learning • PatchTST • Flask Dashboard

1. Title of the invention

ThrashGuard: A PatchTST-Based System for Memory Pressure and Thrashing Classification

2. Field / Area of invention

The project sits at the intersection of operating systems, virtual memory, system-performance monitoring, time-series analysis, and deep learning. Its aim is to classify a machine's memory-related condition from several resource measurements observed over time. The implementation uses Python, PyTorch, Hugging Face Transformers, scikit-learn, Flask, and an HTML-based dashboard.

- Main area: operating-system memory and performance monitoring.
- Learning task: multivariate time-series classification.
- Model family: PatchTST-based Transformer classifier.
- Target environment: Linux/Ubuntu in the intended deployment; the current browser demo uses simulated values.

3. Literature Survey

The references below cover the parts of the project that influence its design: the operating-system meaning of thrashing, ways to observe resource pressure, Transformer-based sequence modelling, patch-based time-series methods, and alternative sequence-classification approaches. They do not describe ThrashGuard as a complete system; each contributes background or a method that can be compared with the prototype.

Reference	Main contribution	Relevance	What it does not cover
Denning (1968) [1]	Explains thrashing and working sets.	OS foundation for memory-management problem.	Not a learned multi-metric classifier.
Linux PSI [2]	Defines CPU, memory, and I/O pressure-stall measurements.	Useful for future live collection and interpreting stalls.	Monitoring interface, not a four-class model.
Vaswani et al. [3]	Introduces Transformer and self-attention.	Sequence-modelling foundation.	Not designed for OS thrashing.
Nie et al. / PatchTST [4]	Represents time series as patches.	Most direct influence on the model family.	Original focus is forecasting and representation learning.
Fawaz et al. [5]	Reviews deep-learning time-series classification.	Positions the approach among CNN and recurrent methods.	Not a thrashing detector.
Bai et al. [6]	Evaluates temporal convolutional networks.	A useful sequence-model baseline for later comparison.	Not OS-specific.
Chuah et al. [7]	Predicts major page faults in cluster systems.	Related work on memory-event prediction.	Different target from four thrashing stages.
Al Maruf & Chowdhury [8]	Studies remote-memory access and prefetching.	Context for memory-access behaviour.	Does not classify thrashing.
Hochreiter & Schmidhuber [9]	Introduces LSTM.	Provides a recurrent-model comparison point.	Different architecture from PatchTST.
Loshchilov & Hutter [10]	Introduces decoupled weight decay / AdamW.	Supports optimizer choice.	Optimization method, not OS monitoring.
Wang et al. [11]	Studies deep neural networks for time-series classification.	An established classification reference.	General time-series task.
Silberschatz et al. [12]	Covers virtual memory and OS memory management.	Textbook background.	Not a learned early-warning method.

Taken together, these sources support the design choices, but they also suggest keeping the claims measured. ThrashGuard is an application of existing OS and time-series ideas to a specific prototype. Its usefulness must be established with realistic traces, careful train/test separation, and comparison against simpler models.

4. Summary and background of the invention (Address the gap / novelty)

Abstract

ThrashGuard is a prototype that classifies memory-related operating-system conditions into Normal, Memory Pressure, Approaching Thrashing, and Thrashing. It uses five measurements—page-fault rate, swap I/O, free memory, CPU usage, and run-queue length—arranged as a sequence of 16 observations. The features are standardized with a fitted StandardScaler and passed to a PatchTST-based Transformer classifier. A Flask backend loads the trained model and scaler and exposes prediction functionality for the dashboard. The supplied dataset contains 800 rows, with 200 rows assigned to each class. In the recorded run, the best test accuracy during training was 75.17%, while the final evaluation reported 68.97%. These results are preliminary because the dataset is small and the browser demo is not yet connected to a live Ubuntu metric collector.

Introduction

Virtual memory allows programs to run without keeping every page they may use in RAM at the same time. When memory demand grows, the operating system may move pages between physical memory and storage. Some paging is normal. The problem becomes serious when page movement consumes so much time that applications make little useful progress; this is commonly called thrashing [1], [12].

A single measurement is not enough to establish that thrashing is happening. Page faults can rise for several reasons, CPU usage can be high during useful work, and free memory can be low without a severe paging loop. ThrashGuard therefore considers several metrics together and looks at a short sequence rather than one isolated row.

Background

A page fault occurs when a process refers to a virtual-memory page that is not currently mapped in physical memory. Not every fault requires storage I/O; major faults are more likely to involve I/O and can be expensive. Swap I/O describes activity involving swap space. Free memory estimates available physical-memory headroom. CPU usage and run-queue length add context about system activity. Linux Pressure Stall Information (PSI) provides another way to measure time lost to resource contention [2].

Rather than applying a hand-written rule such as “if free memory is below X, call it thrashing,” the classifier learns patterns from labelled sequences. This is the intended distinction from a threshold-only demonstration, although a fair comparison against thresholds has not yet been run.

Research Gap

- A high value in one metric can be ambiguous; combining memory and workload measurements may provide a fuller picture than page-fault rate alone.
- Fixed thresholds are easy to interpret, but a threshold applied to one instant may miss the direction and persistence of changing metrics.
- PatchTST is known for patch-based time-series modelling, while related page-fault work studies a different prediction target [4], [7]. This project explores adapting a PatchTST-based classifier to four labelled OS states.

Project-level contribution: the implemented combination of five OS metrics, a 16-step input window, four-class PatchTST-based classification, a saved preprocessing/model pipeline, and a Flask dashboard. This describes the prototype; it does not claim that the architecture or the general idea of memory monitoring is new.

5. Objective(s) of invention

1. Use several OS measurements together to describe memory and workload behaviour.
2. Represent recent behaviour as a fixed-length time series of 16 observations and five features per observation.
3. Train a PatchTST-based classifier for Normal, Memory Pressure, Approaching Thrashing, and Thrashing.

4. Save the trained model and fitted scaler so inference uses the same preprocessing as training.
5. Provide a Flask backend that can return predictions to a browser dashboard.
6. Record prototype performance and identify what must be validated before claiming reliable live detection.

6. Working principle of the invention (in detail)

Metric inputs — Five metrics are considered: page-fault rate, swap I/O, free memory, CPU usage, and run-queue length. These are intended live inputs; the current browser demo uses simulated values.

Data validation — The training script checks dataset dimensions, missing values, class balance, and basic invalid values. The CSV has 800 rows, eight columns, no missing values, and 200 rows per class.

Feature scaling — StandardScaler transforms each feature using the mean and standard deviation calculated from the training portion. The fitted scaler is saved and reused during inference.

Sequence construction — Five features are grouped into windows of 16 consecutive rows, giving a 16 × 5 input matrix for each sample.

Patching — Patch length four and stride four divide each 16-step sequence into four non-overlapping patches.

Transformer encoding — The configured model uses two Transformer encoder layers, model dimension 64, four attention heads, and feed-forward dimension 128.

Classification — The encoded sequence is passed to a classification head, which produces scores for the four target classes.

Training — Cross-entropy loss measures classification error. AdamW updates model parameters. The run used learning rate 0.0005, batch size 32, and 30 epochs.

Saving and inference — The model is saved under models/patchtst and the scaler as models/scaler.pkl. app.py loads these artefacts and serves the prediction interface.

Dashboard — Flask connects the model to the browser interface. The app starts locally, but live Ubuntu metric collection remains future work.

7. Description of the invention in detail

7.1 Input metrics and useful calculations

The five features describe different parts of system behaviour. The definitions below are useful when implementing a live Linux collector. They should not be read as proof that the current dashboard calculates every value from a real machine: the present dashboard uses simulated inputs, and the sampling interval and units must be consistent between collection and training.

Feature	Meaning	Calculation / collection note	Interpretation caution
page_fault_rate	Fault events observed during a sampling interval.	Rate = fault count / interval duration. Example: 240 faults in 2 s = 120 faults/s.	Fault type and workload matter; rate alone does not prove thrashing.
swap_io	Swap activity; collector must define operations or bytes.	If measured in bytes/s: bytes transferred during interval / duration. Example: 8 MB in 2 s = 4 MB/s.	Sustained activity may indicate pressure, but brief activity alone is not enough.
free_memory_mb	Available/free physical memory in a consistent unit.	MiB = bytes / 1,048,576. Example: 1,572,864,000 bytes = 1,500 MiB.	OS definitions and reclaimable cache matter.
cpu_usage	CPU busy percentage over an interval.	CPU % = 100 × (1 − idle-time-delta / total-time-delta). If idle delta is 30 ms of 100 ms total, usage = 70%.	High or low CPU use alone does not diagnose thrashing.
run_queue_length	Runnable tasks waiting for CPU service, as defined by the collector.	Use the same source and sampling method for every observation; do not mix queue count and load average without documenting it.	Indicates contention/workload, not memory pressure directly.

7.2 Preprocessing and sequence calculations

StandardScaler places each feature on a comparable scale. For a feature value x, the standardized value is:

$$z = (x - \mu) / \sigma$$

x = observed value; μ = mean from training data; σ = standard deviation from training data.

For illustration only, if x = 120, μ = 100, and σ = 20, then z = (120 − 100) / 20 = 1.0. The actual scaler values are learned from the dataset; this is not a reported dataset measurement.

One input contains 16 time steps and five features, so it contains 16 × 5 = 80 scalar feature values. With sequence length L = 16, patch length P = 4, and stride S = 4:

$$\text{Number of patches} = \text{floor}((L - P) / S) + 1 = \text{floor}((16 - 4) / 4) + 1 = 4$$

Each patch contains 4 × 5 = 20 scalar values before embedding. The four non-overlapping patches cover the complete 16-step window.

The training log reports 640 raw rows for training and 160 for testing. With a sliding window of length 16 moving one row at a time, possible windows are N − L + 1: 640 − 16 + 1 = 625 training sequences and 160 − 16 + 1 = 145 test sequences, matching the log. For a stronger time-series evaluation, split by chronological trace/session before generating windows to reduce leakage between neighbouring or overlapping windows.

7.3 Model configuration

Parameter	Current implementation
Input window	16 time steps × 5 features
Patch length / stride	4 / 4; four patches per window
Model family	PatchTSTForClassification / PatchTST-based Transformer classifier
Transformer encoder layers	2
Model dimension	64
Attention heads	4
Feed-forward dimension	128
Loss / optimizer	Cross-entropy / AdamW
Learning rate	0.0005
Batch size / epochs	32 / 30
Classes	Normal; Memory Pressure; Approaching Thrashing; Thrashing

7.4 Software architecture

- preprocessing.py — validates/prepares data, scales features, and creates time-series windows.
- train.py — trains the classifier, reports performance, and saves the best model and scaler.
- evaluate.py — evaluates the saved model pipeline and reports classification metrics.
- app.py — loads the trained model and scaler, serves Flask routes, and performs inference.
- templates/index.html — provides the browser interface; the current demo uses mock metric values.

The model and scaler were saved successfully, and the local Flask startup log confirmed that both artefacts loaded. Live metric collection is still a separate implementation task.

7.5 Output states

Label	Class name	Intended interpretation
0	Normal	Measurements resemble examples labelled normal in the training data.
1	Memory Pressure	Measurements resemble a memory-stressed state.
2	Approaching Thrashing	Intermediate/high-pressure class intended as an early-warning stage.
3	Thrashing	Measurements resemble the severe paging/performance-degradation class.

The output is a classification based on patterns learned from the supplied labels. It should not be presented as a definitive OS diagnosis until tested on independent real-machine traces. The current model has four classes and does not include Recovery.

8. Experimental Validation Results

8.1 Dataset and setup

The file thrashguard_augmented_800.csv contains 800 rows and eight columns: timestamp, page_fault_rate, swap_io, free_memory_mb, cpu_usage, run_queue_length, label, and label_name. Five numerical features are model inputs; label is the target class, label_name is its readable form, and timestamp is not one of the five model inputs.

Label	Class name	Rows	Share
0	Normal	200	25%
1	Memory Pressure	200	25%
2	Approaching Thrashing	200	25%
3	Thrashing	200	25%
—	Total	800	100%

The training log reported no missing values, no negative page-fault, swap, free-memory, or run-queue values, and CPU values within 0–100. Balanced class counts are useful, but they do not prove that the dataset represents real machines or all workloads.

8.2 Training results

Measure	Observed result
Raw rows for training / testing	640 / 160
Sequences after windowing	625 / 145
Best test accuracy observed during training	75.17% (epoch 12)
Final reported evaluation accuracy	68.97%
Final training accuracy	82.40% (epoch 30)
Training loss at epoch 30	0.4472
Training schedule	30 epochs; CPU execution

The final accuracy can be checked from the confusion matrix: 28 + 23 + 21 + 28 = 100 correct predictions. Out of 145 test sequences:

Accuracy = (correct predictions / total predictions) × 100 = (100 / 145) × 100 ≈ 68.97%

The 75.17% value is the best test accuracy recorded at an earlier epoch, while 68.97% is the final evaluation result. They are different reported points in the workflow and should not be combined into one claim.

8.3 Classification metrics

Precision, recall, and F1-score provide more detail than accuracy alone. For each class, treat that class as positive and the other classes as negative:

$$\text{Precision} = TP / (TP + FP)$$

$$\text{Recall} = TP / (TP + FN)$$

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

TP means true positives, FP means false positives, and FN means false negatives.

Class	Precision	Recall	F1-score	Support
Normal	0.80	0.80	0.80	35
Memory Pressure	0.64	0.64	0.64	36
Approaching Thrashing	0.55	0.58	0.57	36
Thrashing	0.78	0.74	0.76	38
Macro average	0.69	0.69	0.69	145
Weighted average	0.69	0.69	0.69	145

The model performed best on Normal and had the most difficulty with Approaching Thrashing. That is important because the intermediate class is the one most relevant to an early-warning use case.

8.4 Confusion matrix

Rows show the actual class and columns show the predicted class, in the order Normal, Memory Pressure, Approaching Thrashing, Thrashing.

Actual / predicted	Normal	Memory Pressure	Approaching Thrashing	Thrashing
Normal	28	5	2	0
Memory Pressure	7	23	5	1
Approaching Thrashing	0	8	21	7
Thrashing	0	0	10	28

The diagonal entries total 100 correct predictions. Errors occur between adjacent severity levels: some Memory Pressure examples are labelled Normal, while some Approaching Thrashing and Thrashing examples are confused with one another. The cause needs further investigation using real traces and more detailed error analysis.

8.5 Limitations and further validation

- The dashboard currently uses simulated values; live Ubuntu collection has not yet been validated end to end.
- The dataset contains only 800 rows. More real traces from different machines and workloads are needed.
- The windowing and split procedure should be reviewed to prevent similar or overlapping windows leaking between training and testing. A chronological split by independent trace/session is preferable.
- Compare PatchTST with simple thresholds, logistic regression, random forest, LSTM, and temporal convolutional network baselines.
- Report repeated-run variation, class-specific metrics, false-alarm rate, missed-warning rate, and calibration—not accuracy alone.
- The current four-class model has no Recovery category and does not automatically terminate processes or adjust resource allocation.

These limitations do not prevent the prototype from serving as a project demonstration. They define the next experiments needed before claiming reliable live thrashing detection.

9. References

[1] P. J. Denning, “Thrashing: Its Causes and Prevention,” in Proceedings of the AFIPS Fall Joint Computer Conference, vol. 33, 1968, pp. 915–922, doi: 10.1145/1476589.1476705.

[2] J. Weiner, “PSI — Pressure Stall Information,” Linux Kernel Documentation. Available: https://docs.kernel.org/accounting/psi.html (accessed Oct. 22, 2026).

[3] A. Vaswani et al., “Attention Is All You Need,” in Advances in Neural Information Processing Systems, vol. 30, 2017.

[4] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, “A Time Series Is Worth 64 Words: Long-Term Forecasting with Transformers,” in International Conference on Learning Representations (ICLR), 2023. Available: https://openreview.net/forum?id=Jbdc0vTOcol.

[5] H. I. Fawaz et al., “InceptionTime: Finding AlexNet for Time Series Classification,” Data Mining and Knowledge Discovery, vol. 34, pp. 1936–1962, 2020, doi: 10.1007/s10618-020-00710-y.

[6] S. Bai, J. Z. Kolter, and V. Koltun, “An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling,” arXiv:1803.01271, 2018. Available: https://arxiv.org/abs/1803.01271.

[7] E. Chuah, A. Jhumka, S. Narasimhamurthy, and A. Ayesh, “Deep Learning-Based Prediction of Major Page Faults in Cluster Systems,” CCF Transactions on High Performance Computing, vol. 7, pp. 413–430, 2025, doi: 10.1007/s42514-025-00233-2.

[8] H. Al Maruf and M. Chowdhury, “Effectively Prefetching Remote Memory with Leap,” in Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC ’20), 2020.

[9] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.

[10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in International Conference on Learning Representations (ICLR), 2019. Available: https://openreview.net/forum?id=Bkg6RiCqY7.

[11] Z. Wang, W. Yan, and T. Oates, “Time Series Classification from Scratch with Deep Neural Networks: A Strong Baseline,” in International Joint Conference on Neural Networks (IJCNN), 2017, pp. 1578–1585, doi: 10.1109/IJCNN.2017.7966039.

[12] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ, USA: Wiley, 2018.